pdf4tcl

PDF erzeugen, Formulare, CID, Verschlüsselung

2026-10-25

Inhaltsverzeichnis

1 pdf4tcl verstehen 6
1.1 Einführung 6
1.2 Erstes Dokument 6
1.3 Was pdf4tcl ist — und was nicht 6
1.4 Koordinaten und Einheiten 6
1.5 Warnungen 7
1.6 Verwandte Quellen 7
1.7 Zusammenfassung 7
2 Dokument und Seite 7
2.1 Einführung 7
2.2 Konstruktor 7
2.3 Seite 8
2.4 Schreiben 8
2.5 Rand und Ursprung 8
2.6 Zusammenfassung 9
3 Text und Standardschriften 9
3.1 Einführung 9
3.2 Schrift setzen und schreiben 9
3.3 ToUnicode 9
3.4 Breite und Kasten 9
3.5 PDF/A und Standardschriften 10
3.6 Zusammenfassung 10
4 CID und Unicode 10
4.1 Einführung 10
4.2 Rezept 10
4.3 Zwei Wege, zwei Decken 10
4.4 Ersetzungen prüfen 11
4.5 Tcl 8.6 und Zeichen über U+FFFF 11
4.6 Verschlüsselung 11
4.7 Zusammenfassung 12
5 Grafik 12
5.1 Einführung 12
5.2 Farbe 12
5.3 Formen 12
5.4 Transparenz und Verlauf 12
5.5 Transformation 13
5.6 Zusammenfassung 13
6 Bilder 13
6.1 Einführung 13
6.2 PNG, JPEG, TIFF 13
6.3 Wiederverwenden 14
6.4 Getaggt und PDF/UA 14
6.5 Zusammenfassung 14
7 Canvas-Export 14
7.1 Einführung 14

7.2 Klassisches Canvas 14
7.3 tkpath und tko::path 15
7.4 Schriftabbildung 15
7.5 Text-Baseline 16
7.6 Zusammenfassung 16
8 Formulare erzeugen 16
8.1 Einführung 16
8.2 Textfeld und Schalter 16
8.3 Optionen 17
8.4 Summe (JS im Viewer) 17
8.5 PDF/A und Häkchen 17
8.6 Ebenen und Felder 18
8.7 Zusammenfassung 18
9 fillForms 18
9.1 Einführung 18
9.2 Lesen, füllen, exportieren 18
9.3 Was getForms liefert 18
9.4 Appearance seit 0.9.4.64 19
9.5 NeedAppearances nur bei altem Stream 19
9.6 Werte 19
9.7 Feldbaum (0.9.4.64) 20
9.8 Roundtrip 20
9.9 Dynamisches XFA 20
9.10 Zusammenfassung 20
10 Annotationen 20
10.1 Einführung 20
10.2 Übliche Aufrufe 21
10.3 Was sich selbst zeichnet 21
10.4 Stempel braucht Schrift 21
10.5 Pfeilspitzen 21
10.6 Links 22
10.7 Getaggt 22
10.8 Zusammenfassung 22
11 Verschlüsselung 23
11.1 Einführung 23
11.2 AES-128 23
11.3 AES-256 23
11.4 CID-Schriften: WriteObjectDirect (0.9.4.67) 23
11.5 Prüfen 24
11.6 Zusammenfassung 24
12 catPdf 24
12.1 Einführung 24
12.2 Rezept 24
12.3 Getaggt mischen 25
12.4 Formulare (0.9.4.44) 25
12.5 XRef-Streams 25
12.6 Grenzen 25

12.7 Zusammenfassung 25
13 PDF/A 26
13.1 Einführung 26
13.2 Stufe B 26
13.3 Stufe U (seit 0.9.4.56) 26
13.4 Stufe A (seit 0.9.4.41) 26
13.5 Fallen 27
13.6 Zusammenfassung 27
14 Ebenen 27
14.1 Einführung 27
14.2 Rezept 27
14.3 Optionen (seit 0.9.4.63) 28
14.4 Zählen 28
14.5 Formulare 29
14.6 Zusammenfassung 29
15 Metadaten 29
15.1 Einführung 29
15.2 Setzen 29
15.3 Viewer 29
15.4 Anhänge 30
15.5 Verschlüsselung 30
15.6 catPdf 30
15.7 Zusammenfassung 30
16 gsave und Text 30
16.1 Einführung 30
16.2 Die Falle bis 0.9.4.61 30
16.3 Seit 0.9.4.62 31
16.4 Andere Methoden 31
16.5 Regel 31
16.6 Tagged Text seit 0.9.4.69 31
16.7 Zusammenfassung 31
17 CID-Subset und CMap 32
17.1 Einführung 32
17.2 cmap ist Pflicht 32
17.3 Identity, nicht Unicode 32
17.4 letzteGlyphe ist numGlyphs-1 32
17.5 Prüfsummen (0.9.4.68) 33
17.6 CFF 33
17.7 Zusammenfassung 33
18 Fallstudie: Bericht 33
18.1 Einführung 33
18.2 Schrift und Dokument 33
18.3 Kopf und Kennzahlen 34
18.4 Feld und Stempel 34
18.5 Später füllen 34
18.6 Was bewusst fehlt 35
18.7 Zusammenfassung 35

19 Fallen **35**
19.1 Text und Schrift 35
19.2 Grafik 35
19.3 Canvas 36
19.4 Formulare und Annotationen 36
19.5 Ausgabe 36
20 API **36**
20.1 Paket und Objekt 36
20.2 Text und Schrift 36
20.3 Grafik 37
20.4 Bilder und Canvas 37
20.5 Formular, Annotation, Ebene 37
20.6 Zusammenfügen, Metadaten, PDF/A 38
21 Glossar **38**
21.1 A 38
21.2 B 38
21.3 C 38
21.4 N 38
21.5 O 38
21.6 P 39
21.7 T 39
21.8 W 39
21.9 X 39
22 Fehlermeldungen **39**
22.1 no font set 39
22.2 unknown color: "1" -- expected a name like "red", "#rrggbb", ... 39
22.3 unknown color / invalid ... component 39
22.4 createFontSpecCID: base font '...' not loaded 40
22.5 invalid TTF file checksum 40
22.6 fillForms: no such field(s) 40
22.7 fillForms: "..." is not an option of "..." 40
22.8 unknown form type ... 40
22.9 unknown metadata option "..." 40
22.10 papersize "..." is unknown 40
22.11 rotation ... not a multiple of 90 40
22.12 endLayer without beginLayer / layer still open 40
22.13 Adobe: leere Seite, keine Meldung 40
22.14 qpdf: inflate: incorrect header check 40
22.15 PDF/A: the standard font Helvetica has no embeddable font program 40
22.16 pdf4tcl: linkAdd target(s) never defined with destinationAdd: ... 41
22.17 destination "..." already defined 41
22.18 invalid destination name "...": must match [A-Za-z0-9._-]+ 41
23 Lizenz **41**

1 pdf4tcl verstehen

1.1 Einführung

pdf4tcl schreibt PDF aus Tcl. Eine Klasse, Methoden am Dokumentobjekt, Ausgabe per `write`. Dieses Kapitel legt fest, wofür das Paket taugt und wo andere Schichten ansetzen.

Das Buch beschreibt **0.9.4.69**: tagged Text bleibt in BT/ET. Dazu kommen die Links im Dokument aus **0.9.4.72** (Kapitel Annotationen). CID-Subset mit Identity-CMap und Tabellen-Prüfsummen seit 0.9.4.68, verschlüsselte CID-Schriften über `WriteObjectDirect` seit 0.9.4.67, Appearance-Streams für Stempel und `fillForms`.

1.2 Erstes Dokument

```
package require pdf4tcl 0.9.4.69

set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient true -compress 1]
$pdf startPage
$pdf setFont 12 Helvetica
$pdf text "Hallo Welt" -x 50 -y 50
$pdf endPage
$pdf write -file ausgabe.pdf
$pdf destroy
```

%AUTO% vergibt den Objektnamen. `-orient true` (Default) setzt Y oben, wie Tk. `-compress 1` (Default) packt Content-Streams.

Ohne `startPage` beginnt die erste Zeichenoperation die Seite selbst. Trotzdem `startPage` / endPage explizit setzen: Rand, Papier und offene Ebenen bleiben dann übersichtlich.

1.3 Was pdf4tcl ist — und was nicht

Geeignet:

- Seiten mit Text, Vektoren, Bildern
- Unicode über CID-Schrift
- AcroForm anlegen und später füllen
- Canvas-Export, AES-Verschlüsselung, PDF/A, Ebenen
- Mehrere PDFs aneinanderhängen (`catPdf`)

Ungeeignet als alleinige Schicht:

- Tabellen mit Seitenumbruch, Fließtext über Spalten, Etikettenbogen
- Das leistet `pdf4tcllib` auf derselben Zeichen-API

pdf4tcl zeichnet. Layout bleibt Aufgabe des Aufrufers oder der Begleitbibliothek.

1.4 Koordinaten und Einheiten

Default-Einheit ist Punkt (`-unit p`). Millimeter: `-unit mm`. Mit `-orient 1` liegt der Ursprung oben links im Satzspiegel (nach `-margin`). PDF intern zählt Y von unten; die Klasse rechnet um.

Farben liegen in **0.0.. 1.0**, nicht 0..255. Seit 0.9.4.39 wirft ein Wert außerhalb einen Fehler statt stillschweigend zu schreiben.

1.5 Warnungen

Manche Mängel unterbrechen nicht: Standardschrift unter `-pdfa`, Alpha unter PDF/A-1. Nach `write` die Liste prüfen:

```
if {[llength $::pdf4tcl::warnings]} {
    puts $::pdf4tcl::warnings
}
```

1.6 Verwandte Quellen

- Manpage `pdf4tcl.md` — jede Methode
- `pdf4tcl/doc/en/howtos/` — eine Aufgabe je Datei, mit Skript
- `pdf4tcl/UPGRADING.md` — was sich an bestehendem Code ändert
- `pdf4tcl/demo/` — ein Merkmal je Programm

Dieses Buch fasst den Stoff als Lehrtext. Die Howtos bleiben die ablauffähige Probe.

1.7 Zusammenfassung

- `pdf4tcl::new`, Seite, zeichnen, `write`, `destroy`.
- Version **0.9.4.69** für tagged Text in BT/ET, CID-cmap, AES mit eingebetteter Schrift und Appearance-Streams.
- Layout-Helfer liegen außerhalb dieses Pakets.

Nächstes Kapitel: Dokument und Seite

2 Dokument und Seite

2.1 Einführung

Ein Dokument entsteht im Konstruktor. Jede Seite beginnt mit `startPage` und endet mit endPage. Optionen am Konstruktor gelten als Default; `startPage` darf Papier, Rand und Ausrichtung überschreiben.

2.2 Konstruktor

```
set pdf [::pdf4tcl::new %AUTO% \
        -paper a4 \
        -orient 1 \
        -margin 50 \
        -compress 1]
```

Häufige Optionen:

Option	Default	Bedeutung
-paper	a4	Formatname oder {breite höhe}
-orient	1	Y von oben

Option	Default	Bedeutung
-landscape	0	Breite und Höhe tauschen
-margin	0	einer, zwei oder vier Werte
-unit	p	p, mm, cm, i
-compress	1	Content-Streams packen
-cmyk	0	DeviceCMYK statt RGB
-pdfa	""	Konformitätsstufe, siehe PDF/A
-userpassword	""	siehe Verschlüsselung

Papiergrößen: A-Serie, seit 0.9.4.25 auch B- und C-Serie sowie 4A0 und 2A0. A3-Höhe ist **1191.0** pt (vorher 1190.0). Tests, die die alte Zahl fest eintragen, an `getPaperSize` hängen.

```
lassign [pdf4tcl::getPaperSize a3] w h
lassign [$pdf getPageSize] pw ph
```

`getPageSize` liefert die aktuelle Seite in der Dokumenteinheit.

2.3 Seite

```
$pdf startPage
# zeichnen
$pdf endPage
$pdf startPage -paper a5 -margin 20
# zweite Seite kleiner
$pdf endPage
```

`startPage` bei offener Seite ruft `endPage` selbst. Eine noch offene Ebene (`beginLayer` ohne `endLayer`) lehnt `endPage` ab.

2.4 Schreiben

```
$pdf write -file bericht.pdf
$pdf write -chan $ch
$pdf write
```

`-file` öffnet und schließt die Datei. `-chan` schreibt in einen offenen Kanal (seit 0.9.4.25). Ohne Option geht die Ausgabe nach stdout. Danach `destroy`.

`finish` schließt das Dokument; `write` ruft das intern. Nach `finish` nicht weiterzeichnen.

2.5 Rand und Ursprung

`-margin 50` setzt alle vier Seiten. Zwei Werte: links/rechts, oben/unten. Vier Werte: links, rechts, oben, unten — in der Dokumenteinheit.

Mit `-orient 1` ist `-x 0 -y 0` die obere linke Ecke des Satzspiegels, nicht der Papierecke. Ohne Rand fallen beide zusammen.

2.6 Zusammenfassung

- Konstruktor setzt Defaults; `startPage` darf abweichen.
- `-orient 1`: Y oben. Einheit Default Punkt.
- `write -file` oder `write -chan`, dann `destroy`.

Nächstes Kapitel: Text und Standardschriften

3 Text und Standardschriften

3.1 Einführung

Vierzehn Standardschriften (Base-14) braucht niemand einzubetten: Helvetica, Times, Courier je in vier Schnitten, plus Symbol und ZapfDingbats. Der Zeichenvorrat ist WinAnsi / CP1252. Alles außerhalb wird?.

3.2 Schrift setzen und schreiben

```
$pdf setFont 18 Helvetica-Bold
$pdf text "Titel" -x 0 -y 24
$pdf setFont 12 Times-Roman
$pdf text "Fließtext mit Umlauten: äöü ÄÖÜ ß" -x 0 -y 50
$pdf setFont 10 Courier
$pdf text "mono 0123456789" -x 0 -y 70
```

Schnitte genau so nennen: Helvetica **-Oblique**, Times **-Italic**. `Helvetica-Italic` gibt es nicht.

`text` lässt das Textobjekt (BT … ET) absichtlich offen, damit mehrere Aufrufe einen Block teilen. Andere Zeichenmethoden schließen ihn. Details: gsave und Text.

3.3 ToUnicode

Seit 0.9.4.9 tragen Standardschriften eine ToUnicode-CMap für WinAnsi. Kopieren und `pdftotext` liefern lateinische Umlaute korrekt. Griechisch und Kyrillisch bleiben? — dafür CID und Unicode.

`getSubstCount` zählt Ersetzungen — **unter Tcl 9**. Unter 8.6 substituiert `encoding convertto` still, der Zähler bleibt 0. Die Seite sieht in beiden Generationen gleich aus.

3.4 Breite und Kasten

```
set w [$pdf getStringWidth "Rechnung 2026"]
set w [$pdf getStringWidth "Probe" -font Helvetica -size 12]
```

Ohne vorheriges `setFont` die Variante mit `-font` / `-size` (seit 0.9.4.23).

```
$pdf drawTextBox 0 80 400 120 $absatz -align left -newyvar nextY
$pdf line 0 $nextY 400 $nextY
```

`-newyvar` liefert Y in den Koordinaten des Aufrufers. Vor 0.9.4.44 kam bei `-orient 1` eine

interne Seitenkoordinate; wer damals `pageHeight - $nextY` gerechnet hat, muss die Korrektur entfernen.

Weiches Trennzeichen `U+00AD`: seit 0.9.4.61 Vorschlag, kein Glyph. `text`, `drawTextBox` und `getStringWidth` behandeln es gleich. `Sil\u00ADben` ist so breit wie `Silben`.

3.5 PDF/A und Standardschriften

Base-14 hat kein einbettbares Fontprogramm. PDF/A und PDF/UA verlangen eines. Seit 0.9.4.41 steht das in `::pdf4tcl::warnings`; die Datei entsteht trotzdem und fällt in veraPDF durch. Archiv: TrueType laden, siehe nächstes Kapitel.

3.6 Zusammenfassung

- Base-14 ohne Einbettung, nur Latin-1.
- Schnitte: `-Oblique` bei Helvetica, `-Italic` bei Times.
- `drawTextBox -newyvar` in Aufrufer-Koordinaten (0.9.4.44).
- Weiches Trennzeichen seit 0.9.4.61 unsichtbar und ohne Breite.

Nächstes Kapitel: CID und Unicode

4 CID und Unicode

4.1 Einführung

`createFont` mit WinAnsi deckt Latin-1. Polnisch, Griechisch, Kyrillisch, Mathematik oder CJK brauchen eine CID-Schrift: TrueType laden, CID-Spezifikation anlegen, mit `setFont` wählen.

4.2 Rezept

```
pdf4tcl::loadBaseTrueTypeFont BaseDejaVu \
        /usr/share/fonts/truetype/dejavu/DejaVuSans.ttf
pdf4tcl::createFontSpecCID BaseDejaVu Uni

set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 50]
$pdf startPage
$pdf setFont 12 Uni
$pdf text "Grüße aus München, Polska, Ellada" -x 0 -y 24
$pdf endPage
$pdf write -file unicode.pdf
$pdf destroy
```

Drei Schritte: Basis laden, CID-Namen binden, diesen Namen in `setFont` verwenden. Ohne geladene Basis:

```
createFontSpecCID: base font '…' not loaded (use loadBaseTrueTypeFont first)
```

Farb-Bitmap-Emoji (CBDT/CBLC) und COLR lehnt der Lader ab. Nur Outline-Schriften.

4.3 Zwei Wege, zwei Decken

Weg	Aufruf	Decke
kodierte Teilmenge	createFont / createFontSpecEnc	höchstens 256 Codepunkte
CID	createFontSpecCID	der benutzte Vorrat, Identity-H

Seit 0.9.4.57 **subsetten beide Wege**. Vorher wanderte bei CID die ganze Datei ins PDF. Dateigröße für eine Zeile Griechisch in DejaVu Sans:

Variante	Datei
Teilmenge, 5 Codepunkte	8 860 Byte
Teilmenge, 200 Codepunkte	29 761 Byte
CID nach Subset	50 004 Byte
CID vor 0.9.4.57 (ganze Face)	386 728 Byte

CFF/OpenType bleibt vollständig; pdf4tcl subsettet TrueType. Dann steht eine Warnung in `::pdf4tcl::warnings`.

Bei kodierter Teilmenge? (63) in die Liste aufnehmen. Zeichen außerhalb fallen sonst auf Slot 0 — den ersten angegebenen Codepunkt. Eine Teilmenge, die mit `G` beginnt, macht aus Griechisch `GGGGGG`.

4.4 Ersetzungen prüfen

```
$pdf text $zeile -x 0 -y 40
puts [$pdf getSubstCount]
```

`0` heißt: jedes Zeichen hatte ein Glyph. Fragezeichen auf der Seite ohne Fehler: die Face hat den Codepunkt nicht, oder es lief Base-14.

4.5 Tcl 8.6 und Zeichen über U+FFFF

Als Literal in der Quelle:

	string length	erstes Zeichen
Tcl 9	1	U+1F600
Tcl 8.6	1	U+FFFD

Aus einer UTF-8-Datei lesen beide Generationen korrekt. Keine `\U`-Escapes über U+FFFF in Code, der unter 8.6 laufen muss. Breite mit `getStringWidth` messen, nicht mit `string length`.

4.6 Verschlüsselung

Eingebettete CID-Schriften in einem AES-Dokument brauchen seit **0.9.4.67** denselben Schreibweg wie andere Objekte (`WriteObjectDirect`). Dateien aus 0.9.4.66 neu erzeugen. Siehe Verschlüsselung.

Die Subset-Datei selbst trägt seit **0.9.4.68** eine gültige cmap und Tabellen-Prüfsummen.

Vertiefung: CID-Subset und CMap.

4.7 Zusammenfassung

- `loadBaseTrueTypeFont`, dann `createFontSpecCID`, dann `setFont`.
- CID subsettet TrueType seit 0.9.4.57; CFF bleibt ganz.
- Kodierte Teilmenge: 256 Codepunkte,? in die Liste.
- `getSubstCount` nach dem Zeichnen.

Nächstes Kapitel: Grafik

5 Grafik

5.1 Einführung

Linien, Flächen, Farbe, Transparenz und Verläufe laufen über Methoden am Dokument. Farbe teilt seit 0.9.4.39 eine Pipeline: Füllung, Strich, Verlauf, Formular, Annotation.

5.2 Farbe

```
$pdf setFillColor 1 0 0
$pdf setFillColor #cc3300
$pdf setFillColor red
$pdf setFillColor 0 0.5 1 0
$pdf setStrokeColor navy
$pdf setBgColor 1 1 0
```

Komponenten **0.0.. 1.0**. `255 0 0` ist ein Fehler seit 0.9.4.39 (vorher schrieb die Datei den illegalen Operanden, Viewer klemmten still). 147 X11/CSS-Namen ohne Tk. Namen außerhalb der Tabelle fragen `winfo rgb`, wenn Tk da ist.

`setBgColor` gilt nur für `text -bg`, nicht für die Seite. Seitengrund: vollflächiges Rechteck zuerst zeichnen.

5.3 Formen

```
$pdf setStrokeColor 0 0 0
$pdf setLineWidth 1
$pdf line 0 36 250 36

$pdf setFillColor 0.9 0.9 0.9
$pdf rectangle 0 50 250 60 -filled 1
```

Weitere Methoden: `circle`, `oval`, `polygon`, `arc`, `roundedRect`, `arrow`. Nach farbigem Text die Füllfarbe wieder auf Schwarz setzen, sonst erbt der nächste Kasten Rot.

5.4 Transparenz und Verlauf

```
$pdf setAlpha 0.5
$pdf rectangle 40 40 120 80 -filled 1
$pdf setAlpha 1
```

Unter `-pdfa 1b` / `1a` verletzt Alpha unter 1 PDF/A-1. Seit 0.9.4.22 steht eine Warnung in `::pdf4tcl::warnings`; die Datei entsteht trotzdem.

```
$pdf gsave
$pdf clip 50 100 400 60
$pdf linearGradient 50 130 450 130 red blue
$pdf grestore
$pdf rectangle 50 100 400 60
```

Radial: Innenmittelpunkt/Radius, Außenmittelpunkt/Radius, zwei Farben. `-extend {0 0}` stoppt am Ende. Mit `-cmyk 1` schreibt die Shading `/DeviceCMYK`.

```
$pdf setBlendMode Multiply
# überlappende Füllungen
$pdf setBlendMode Normal
```

5.5 Transformation

```
$pdf gsave
$pdf translate 200 300
$pdf rotate 45
$pdf line 0 0 80 0
$pdf grestore
```

`rotate`, `scale`, `translate` (seit 0.9.4.20) setzen `cm` und merken `rawcoords`. Immer mit `gsave` / `grestore` paaren.

Text folgt cm nicht. `text` setzt `Tm` absolut. Gedrehter Text: trotzdem `translate` / `rotate`, dann `text` bei `-x 0 -y 0` — und vorher gsave und Text lesen: `gsave` schließt ein offenes `BT` .

5.6 Zusammenfassung

- Farbe 0..1, Namen ohne Tk, Bereichsprüfung seit 0.9.4.39.
- Verlauf clippen, dann `linearGradient` / `radialGradient`.
- Transformationen immer zwischen `gsave` und `grestore`.

Nächstes Kapitel: Bilder

6 Bilder

6.1 Einführung

Raster liegt in zwei Schritten auf der Seite: `addImage` nimmt die Datei ins Dokument, `putImage` platziert sie. Dieselbe ID darf mehrfach stehen.

6.2 PNG, JPEG, TIFF

```
set id [$pdf addImage /pfad/foto.png]
$pdf addImage $pfad -id logo
$pdf putImage $id 50 100 -width 200
```

Ohne `-height` folgt die Höhe dem Seitenverhältnis. `-id` vergibt einen festen Namen statt einer Zahl.

JPEG und viele PNG/TIFF-Varianten laufen. Seit **0.9.4.28** wird Adam7-interlaced PNG beim Einbetten entflochten. Interlaced PNG unter 8 Bit lehnt der Lader mit klarer Meldung ab.

Nicht unterstütztes Format: über Tk/Img laden, dann `addRawImage` / `putRawImage`.

6.3 Wiederverwenden

```
set logo [$pdf addImage logo.png -id logo]
$pdf putImage logo 20 20 -width 80
$pdf startPage
$pdf putImage logo 20 20 -width 80
```

Die Bytes stehen einmal im PDF. Jede Platzierung ist ein `Do` auf dasselbe XObject.

6.4 Getaggt und PDF/UA

Abbildungen brauchen ein Strukturelement mit `-alt`:

```
$pdf tagBegin Figure -alt "Firmenlogo"
$pdf putImage $id 50 100 -width 120
$pdf tagEnd
```

Ein Form-XObject mit getaggtem Inhalt darf nur **einmal** gezeichnet werden (0.9.4.46). Unter PDF/UA verbietet veraPDF Regel 7.20-2 jede weitere Platzierung; pdf4tcl warnt dann, bricht aber nicht ab. Wiederverwenden bleibt gutes PDF — die Konformitätsbehauptung schließt es aus.

6.5 Zusammenfassung

- `addImage`, dann `putImage` mit Breite oder Höhe.
- Interlaced PNG seit 0.9.4.28; Sub-8-Bit interlaced bleibt Fehler.
- PDF/UA: `Figure` mit `-alt`; XObject nicht mehrfach setzen, wenn UA behauptet wird.

Nächstes Kapitel: Canvas-Export

7 Canvas-Export

7.1 Einführung

Methode canvas am Dokument zeichnet ein Tk-Widget auf die Seite. Vorher `update`, damit Layout und Items stehen. Drei Widget-Klassen, drei Codepfade.

Braucht **wish** bzw. gesetztes `DISPLAY`.

7.2 Klassisches Canvas

```
package require Tk
package require pdf4tcl 0.9.4.69

wm withdraw .
canvas .c -width 400 -height 300 -background white
pack .c
.c create rectangle 10 10 120 80 -fill #cde -outline black
.c create text 65 45 -text "Probe" -anchor center
update idletasks

set pdf [pdf4tcl::new %AUTO% -paper a4 -orient true]
$pdf startPage
set bb [.c bbox all]
$pdf canvas .c -bbox $bb -x 50 -y 50 -width 400 -height 300
$pdf endPage
$pdf write -file canvas.pdf
$pdf destroy
```

`-bbox` ist der Quellbereich im Canvas. `-x` / `-y` / `-width` / `-height` liegen auf der PDF-Seite. `-sticky` steuert das Einpassen (nw Default). Weitere Optionen: `-bg`, `-fontmap`, `-textscale`.

Ohne `update` ist `bbox` oft leer oder veraltet. Leere bbox: die Methode kehrt still zurück, die Seite bleibt leer.

7.3 tkpath und tko::path

Dieselbe Methode: `$pdf canvas .path …`. Bei PathCanvas und TkoPath ist `-bbox` **Pflicht**.

Die Matrix-Form unterscheidet sich:

Widget	Matrix
tkpath	verschachtelt
tko::path	flach {a b c d tx ty}

Die Formen nicht mischen. pdf4tcl erwartet für tko die flache Form.

window-Items werden oft übersprungen. Photo vorher per `addImage` anmelden, wenn der Export das Bild sonst nicht findet.

7.4 Schriftabbildung

Text außerhalb Latin-1: TrueType unter dem **Tk-Familiennamen** laden (seit 0.9.4.60), dann kein `-fontmap`:

```
pdf4tcl::loadBaseTrueTypeFont Base tahoma.ttf
pdf4tcl::createFontSpecCID Base Tahoma
.c create text 20 30 -text "Grüße" -font {Tahoma 14}
$pdf canvas .c -bbox [.c bbox all]
puts [$pdf getSubstCount]
```

Wo der Name nicht frei wählbar ist, `-fontmap` setzen. Die Abbildung gewinnt immer gegen die Familien-Suche.

Der Schlüssel ist nicht derselbe:

- `tk::canvas`: **ganze** Font-Spezifikation — `{Helvetica 14}` liegt nicht unter `Helvetica`. Ein benannter Font trifft sicherer.
- `tko::path`: nur die **Familie**.

Falscher Schlüssel: Fragezeichen, kein Fehler. Deshalb `getSubstCount`.

7.5 Text-Baseline

PDF-Text sitzt auf der **Baseline**, Canvas-Text am `-anchor`. Ein sichtbarer Versatz in Y ist erwartet, wenn Anker `nw` und PDF-Text ohne Korrektur gemischt werden. Innerhalb von `$pdf canvas` übernimmt pdf4tcl die Item-Anker.

7.6 Zusammenfassung

- `update`, dann `bbox`, dann `$pdf canvas`.
- Drei Klassen: `canvas`, PathCanvas, TkoPath.
- tko: flache Matrix. tkpath: verschachtelt.
- `-fontmap` oder Familienname; `getSubstCount` prüfen.

Nächstes Kapitel: Formulare erzeugen

8 Formulare erzeugen

8.1 Einführung

`addForm` legt AcroForm-Felder auf die aktuelle Seite. Der Appearance-Stream entsteht beim Anlegen. Ein Dokument, das in einem Durchgang mit Werten gebaut wird, braucht `fillForms` nicht.

Feldtext ist praktisch Base-14 / Latin-1. CID in AcroForm ist hart; Umlaute in Beschriftungen auf der Seite als normaler `text` setzen.

8.2 Textfeld und Schalter

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 50]
$pdf startPage
$pdf setFont 12 Helvetica
$pdf text "Name:" -x 0 -y 40
$pdf addForm text 60 28 200 16 -id f_name

$pdf addForm pushbutton 0 70 90 20 -id f_reset \
        -caption "Reset" -action reset
$pdf addForm pushbutton 100 70 90 20 -id f_go \
        -caption "Submit" -action submit \
        -url "mailto:bestellung@example.com"
$pdf endPage
$pdf write -file formular.pdf
$pdf destroy
```

Typen: `text`, `password`, `checkbutton` (Alias `checkbox`), `radiobutton`, `combobox`,

`listbox`, `pushbutton`, `signature`.

`-id` nur Wortzeichen, kein Punkt. Verschachtelte Namen entstehen im Feldbaum fremder Dateien, nicht beim Anlegen hier.

8.3 Optionen

Option	Wirkung
-init	Startwert, überall sichtbar
-align	left / center / right
-multiline	mehrzeilig
-maxlen / -comb	Kammfeld, Zellen = Breite / MaxLen
-options	Auswahlliste als Labels (kein Export/Label-Paar)
-tooltip	Kurzinfo
-readonly / -required	Flags
-color -bgcolor -bordercolor -borderwidth	Darstellung
-layer	/OC an der Annotation
-format date	Formatierung im Viewer
-calculate	Summe über Feld-IDs, braucht JS im Viewer

`-comb 1` verlangt `-maxlen`, schließt `-multiline` aus, ehrt `-align`. Seit 0.9.4.63.

`text`, `password`, `checkbutton` und `radiobutton` laufen vor `setFont`. `combobox`, `listbox`, `pushbutton` und `signature` verlangen eine gesetzte Schrift (Meldung seit 0.9.4.63: `no font set`).

Leeres Textfeld trägt seit 0.9.4.63 einen Appearance-Stream (Länge 0), damit `fillForms` etwas überschreiben kann.

Leeres Feld mit `-bordercolor` und `-borderwidth` zeichnet seit 0.9.4.66 den Rahmen statt eines leeren Streams. Ohne Rahmen bleibt der Stream leer.

8.4 Summe (JS im Viewer)

```
$pdf addForm text 400 200 90 16 -id b1 -align right -init 120
$pdf addForm text 400 220 90 16 -id b2 -align right -init 80
$pdf addForm text 400 250 90 16 -id f_sum -align right \
        -calculate {sum {b1 b2}} -init 200 \
        -borderwidth 1 -bgcolor {0.95 0.95 0.85}
```

`-init` zeigt den Wert überall. `-calculate` aktualisiert in Acrobat, Firefox, Chromium, Foxit.

8.5 PDF/A und Häkchen

Ein Häkchen als ZapfDingbats-Glyph ist nicht einbettbar. Seit 0.9.4.55 zeichnet pdf4tcl den Haken als Vektor, sobald `-pdfa` oder PDF/UA gesetzt ist. Ohne Konformitätsanspruch bleibt der Glyph. `-markstyle font` / `vector` / `auto` (Default `auto`).

8.6 Ebenen und Felder

`beginLayer` erreicht Annotationen nicht — sie liegen außerhalb des Content-Streams. `-layer` an `addForm` schreibt `/OC` ins Annotations-Dictionary.

8.7 Zusammenfassung

- `addForm TYP x y b h -id name`.
- Appearance entsteht beim Anlegen.
- Comb braucht `-maxlen`. Schriftpflicht hängt am Typ.
- PDF/A: Häkchen als Vektor (`-markstyle auto`).

Nächstes Kapitel: fillForms

9 fillForms

9.1 Einführung

Drei Aufrufe arbeiten auf einer **fertigen Datei**, nicht auf dem Dokumentobjekt: `getForms`, fillForms, `exportForms` (seit 0.9.4.50).

9.2 Lesen, füllen, exportieren

```
set fields [pdf4tcl::getForms "auftrag.pdf"]
pdf4tcl::fillForms "auftrag.pdf" "ausgefuellt.pdf" {f_name "Meier & Co"}
pdf4tcl::exportForms "ausgefuellt.pdf" "ausgefuellt.fdf"
```

`fillForms` schreibt Werte und gibt zurück, wie viele Felder es getroffen hat. Unbekannter Name ist ein Fehler:

```
fillForms: no such field(s) in "auftrag.pdf": kein_feld
```

Nicht genannte Felder behalten, was sie hatten.

Verschlüsselte Dateien: `getForms` liefert leer, `fillForms` findet keine Felder — es gibt kein `-password`. Vor dem Füllen unverschlüsselt bauen oder die Kennwörter weglassen.

9.3 Was getForms liefert

Dict von Feld-ID auf Dict:

Schlüssel	Bedeutung
type	/Tx, /Btn, /Ch, /Sig
value	aktueller Wert, entpackt

Schlüssel	Bedeutung
flags	Feldflags
default	Appearance-Zustand bei Buttons
maxlen	Maximallänge oder leer (0.9.4.63)
comb	1 bei Kammfeld (0.9.4.63)
widgets	Widget-Anzahl (0.9.4.64)
description	Tooltip (0.9.4.65)
options	erlaubte Werte der Auswahl (0.9.4.65)

Code, der das ganze Dict mit einem Literal vergleicht, merkt die neuen Schlüssel.

9.4 Appearance seit 0.9.4.64

`fillForms` schreibt den Wert **und** baut den Stream neu — für einfaches Textfeld, mehrzeilig und Comb **mit** `/MaxLen`. Ein Stream pro Widget, aus dessen eigenem Rechteck.

Bis 0.9.4.63 blieb der Stream: ein Viewer mit Flagge zeigte den neuen Wert, der Druckpfad den **alten**. Leeres Feld: nichts auf dem Papier.

Nicht neu gebaut, absichtlich:

Feld	Stream
Comb ohne /MaxLen	alt — kein Teiler, keine Zellen
Kennwort	alt — sonst Klartext im Stream
Auswahl (choice)	alt — Kasten und Rahmen gingen verloren
Check / Radio	Umschalten der vorhandenen Zustände

9.5 NeedAppearances nur bei altem Stream

NeedAppearances setzt fillForms **nur**, wo der Stream alt bleibt (seit 0.9.4.66). Die Flagge sagt dem Viewer: bau alles neu.

Beobachtet: Ein Textfeld zu füllen ließ ein **unberührtes** Kästchen seinen Rahmen verlieren. PDFium baut unter der Flagge jede Off-Erscheinung neu und liefert eine leere.

Wo der Stream selbst gezeichnet wurde, bleibt die Flagge weg. Wo Comb ohne MaxLen, Kennwort oder Auswahl den alten Stream behalten, steht sie — sonst sähe man den neuen Wert nirgends.

Wer sich darauf verlassen hat, dass die Flagge immer da ist: Feld des alten Typs füllen oder die Flagge selbst setzen.

9.6 Werte

Text: Zeichenkette. Check / Radio: Zustandsname **mit Schrägstrich**, wie in der Datei:

```
pdf4tcl::fillForms in.pdf out.pdf {agreed /Yes}
```

Auswahl: Exportwert oder Label. Alles andere ist Fehler (seit 0.9.4.65). Leerer Wert ist erlaubt: nichts gewählt.

```
fillForms: "Frankreich" is not an option of "land";
allowed: Deutschland Niederlande Belgien
```

Bis 0.9.4.64 wurde jeder Wert geschrieben und Erfolg gemeldet; das Feld trug dann einen Zustand, den kein Viewer zeigt.

9.7 Feldbaum (0.9.4.64)

`getForms` und `fillForms` laufen über den Baum, nicht über jedes Widget einzeln. Ein Feld auf mehreren Blättern (Frachtbrief) wird gefunden. Verschachtelter Name: `person.city`, nicht `city`. Wer früher den Kurznamen füllte, trifft nicht mehr.

9.8 Roundtrip

`getForms` gibt Text **entpackt** zurück, genau so wie `fillForms` ihn nimmt (seit 0.9.4.55):

```
set fields [pdf4tcl::getForms "in.pdf"]
set values {}
dict for {id info} $fields {
    dict set values $id [dict get $info value]
}
dict set values f_name "Meier & Co (GmbH)"
pdf4tcl::fillForms "in.pdf" "out.pdf" $values
```

Name-Werte (`/Yes`, `/Off`) bleiben, wie fillForms sie für Buttons erwartet.

9.9 Dynamisches XFA

Dynamische XFA-Formulare lehnt fillForms ab (0.9.4.64): die Seiten entstehen aus XML, `/V` ändert nichts Sichtbares. Hybride XFA (XFA plus nutzbare AcroForm-Felder) werden wie bisher gefüllt.

9.10 Zusammenfassung

- Appearance für einfaches Textfeld seit 0.9.4.64 neu.
- Comb ohne MaxLen, Kennwort, Auswahl: alter Stream.
- NeedAppearances nur dort, wo der Stream alt bleibt.
- Namen: volle Kette, `person.city`.

Nächstes Kapitel: Annotationen

10 Annotationen

10.1 Einführung

Notizen, Stempel, Markierungen und Linien liegen **neben** dem Content-Stream. Ein Name ist

keine Zeichnung. Ob ein Viewer etwas sieht, hängt vom Typ und vom Appearance-Stream ab.

10.2 Übliche Aufrufe

```
$pdf addAnnotNote 100 100 20 20 -content "Prüfen" -author "Lektor" \
        -icon Comment -color {0.6 0.8 1.0}
$pdf addAnnotFreeText 50 200 200 40 "Immer sichtbar" \
        -color {0 0 0} -bgcolor {1 1 0.8}
$pdf addAnnotStamp 300 500 80 30 -name Approved -color {1 0 0}
$pdf addAnnotHighlight 50 300 200 14 -color {1 1 0}
$pdf addAnnotUnderline 50 320 200 14
$pdf addAnnotStrikeOut 50 340 200 14 -color {1 0 0}
$pdf addAnnotLine 50 400 200 400 -color {0 0 0}
```

10.3 Was sich selbst zeichnet

Art	Gezeichnet aus
Highlight, Underline, StrikeOut	Fläche und Farbe
FreeText	Text und Farben
Note	Icon-Satz des Viewers
Stamp	nur Appearance-Stream
Line	nur Appearance-Stream

Ein Stempel nennt, welcher Stempel gemeint war. Acrobat hat Artwork für die benannten Stempel. **PDFium hat keines**, und PDFium ist der Viewer in Chrome und Edge. Dasselbe gilt für die Linie: Endpunkte sagen wo, nicht wie.

Seit **0.9.4.64** schreiben `addAnnotStamp` und `addAnnotLine` einen Appearance-Stream. Geprüft an `demo/demo-annotations.pdf`: keiner von sechs Stempeln und fünf Linien war in PDFium sichtbar, alle elf sind es jetzt. Die Datei wird etwas größer.

10.4 Stempel braucht Schrift

Die Erscheinung eines Stempels nennt eine Schrift. Deshalb wird der Stream **nur nach setFont** geschrieben. Ohne gesetzte Schrift entsteht die Annotation wie vor 0.9.4.64 — der Aufruf schlägt nicht fehl, PDFium bleibt leer.

Reihenfolge:

```
$pdf setFont 10 Helvetica
$pdf addAnnotStamp 300 500 80 30 -name Approved -color {1 0 0}
```

10.5 Pfeilspitzen

Gezeichnet für `OpenArrow`, `ClosedArrow`, `ROpenArrow`, `RClosedArrow`, `Circle`, `Square`. Jedes andere Ende bleibt eine glatte Linie. Eine Linie ohne Kopf ist eine halbe Antwort, ein falsch gezeichneter Kopf eine falsche.

10.6 Links

Ein Link ist eine Annotation vom Typ `/Link`: ein Rechteck, das beim Klick etwas auslöst. Rahmen gibt es nur auf Wunsch.

hyperlinkAdd öffnet eine Adresse (`/S /URI`):

```
$pdf hyperlinkAdd 50 100 200 20 "https://www.tcl-lang.org"
```

Sprünge **im Dokument** gibt es seit **0.9.4.72**. Zwei Methoden gehören zusammen. destinationAdd legt auf der aktuellen Seite ein benanntes Ziel an; linkAdd setzt ein Rechteck, das zu diesem Namen springt:

```
$pdf startPage
$pdf text "1 Einleitung ........ 3" -x 50 -y 100
$pdf linkAdd 50 88 200 14 kap1          ;# Ziel gibt es noch nicht
$pdf endPage
...
$pdf startPage
$pdf text "1 Einleitung" -x 50 -y 60
$pdf destinationAdd kap1 -y 45          ;# Ziel, etwas über der Zeile
```

Der Link trägt nur den Namen (`/Dest /kap1`). Aufgelöst wird er erst beim Schreiben über das Wörterbuch `/Dests` im Katalog. Deshalb darf das Ziel **nach** dem Link entstehen — genau das braucht ein Inhaltsverzeichnis, das vor seinen Kapiteln steht.

- Name: Buchstaben, Ziffern, `. _ -`; je Dokument einmal.
- `-y` (und `-x`) in der aktuellen Einheit; ohne `-y` die Oberkante der Seite.
- Rahmen und Hervorhebung wie bei `hyperlinkAdd`: `-borderwidth -bordercolor -borderradius -borderdash -highlight`.
- Ein Link auf einen Namen, den kein `destinationAdd` anlegt, bricht nichts ab. Die Datei entsteht, der Link bleibt wirkungslos, und stderr meldet den Namen.
- Unter `-pdfa 2b` erlaubt.

Geprüft mit `tests/linkdest.test` (12 Fälle) und an docir-Büchern: das Inhaltsverzeichnis des macOS-Buchs, 238 Einträge, jeder springt auf die gedruckte Seite und dort auf die Überschrift.

10.7 Getaggt

Annotationen gehören zum Strukturbaum nur in offenem `tagBegin Link` oder `tagBegin Annot` … `tagEnd`. Sonst klickbar, für Hilfstechnik unsichtbar. Seit 0.9.4.39 steht das einmal pro Dokument in `::pdf4tcl::warnings`. Für PDF/UA die Annotation einwickeln.

10.8 Zusammenfassung

- Stempel und Linie brauchen Appearance (0.9.4.64), sonst PDFium leer.
- Vor `addAnnotStamp setFont`.
- Highlight zeichnet sich aus Fläche und Farbe.
- `hyperlinkAdd` für Adressen, `destinationAdd` + `linkAdd` für Sprünge im Dokument (0.9.4.72); das Ziel darf nach dem Link kommen.

Nächstes Kapitel: Verschlüsselung

11 Verschlüsselung

11.1 Einführung

Kennwort am Konstruktor schaltet AES ein. AES-128 (`-encversion 4`, Default seit 0.9.4.11) und AES-256 (`-encversion 5`, seit 0.9.4.16). Ohne Kennwort bleibt die Datei offen.

11.2 AES-128

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 \
        -userpassword "oeffnen" \
        -ownerpassword "aendern" \
        -permissions print]
$pdf startPage
$pdf setFont 12 Helvetica
$pdf text "Vertraulich." -x 50 -y 50
$pdf endPage
$pdf write -file geheim.pdf
$pdf destroy
```

Leeres `-ownerpassword`: das User-Kennwort gilt für beide. `-permissions` nimmt symbolische Listen, Presets oder eine rohe `/P`-Zahl. Default wirkt als "alles erlaubt".

Kennwort	Wirkung
user	öffnet; /P gilt
owner	volle Rechte; /P ignoriert

`-permissions` allein sperrt die Datei **nicht**. Ohne User-Kennwort öffnet jeder Viewer; `/P` ist Hinweis an konforme Leser.

11.3 AES-256

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 \
        -userpassword "oeffnen" -ownerpassword "aendern" \
        -encversion 5]
```

Braucht ein SHA-384-Backend. Rein-Tcllib ist langsam (Sekunden statt Millisekunden). In Produktion ein schnelles Krypto-Paket vorziehen.

11.4 CID-Schriften: WriteObjectDirect (0.9.4.67)

Bis **0.9.4.66** liefen die fünf Objekte von `WriteCIDFontObjects` am Encrypt vorbei gerade in die Datei. Das Dokument entstand, war ohne Kennwort dicht — und **mit** Kennwort unlesbar:

```
qpdf: inflate: incorrect header check
pdftotext -upw … : (leer)
```

Der Font-Stream war bytegleich zur unverschlüsselten Datei. Ein Leser, der `/Encrypt`

gehorcht, entschlüsselt ihn trotzdem und erhält Müll.

Seit **0.9.4.67** schreibt `WriteObjectDirect` dieselben Objekte über Encrypt (Strings und Stream, Schlüssel an der Objektnummer). Workarounds (Standardschrift in verschlüsselter Ausgabe) können weg. Dateien aus .66 neu erzeugen — der Inhalt steht im Klartext in der alten Datei, der eingebettete Font ist für den Leser tot.

XMP-Metadaten liefen bis .66 ebenfalls im Klartext, während die Datei `/EncryptMetadata true` behauptet. `qpdf --check` fängt das nicht: der Stream ist unkomprimiert, es gibt kein inflate.

11.5 Prüfen

```
qpdf --password=oeffnen --check geheim.pdf
pdftotext geheim.pdf -                 # Incorrect password
pdftotext -upw oeffnen geheim.pdf -    # Text
```

Alle drei zählen. Die ersten zwei allein bestehen auch bei einer verschlüsselten, aber kaputten Datei — genau das, was 0.9.4.66 erzeugte.

11.6 Zusammenfassung

- AES-128: `-encversion 4`. AES-256: `-encversion 5`.
- User-Kennwort zum Öffnen, Owner für volle Rechte.
- CID plus Encrypt: erst ab 0.9.4.67 lesbar. Alte Dateien neu bauen.

Nächstes Kapitel: catPdf

12 catPdf

12.1 Einführung

catPdf hängt fertige PDFs aneinander. Kein Dokumentobjekt. Seit 0.9.4.40 bleibt die logische Struktur erhalten. Seit 0.9.4.44 werden Formulare vereinigt.

12.2 Rezept

```
package require pdf4tcl 0.9.4.69
pdf4tcl::catPdf teil1.pdf teil2.pdf gesamt.pdf
```

Optionen **vor** den Dateinamen (seit 0.9.4.48):

```
::pdf4tcl::catPdf -title "Gesamtdatei" -author "" \
        teil1.pdf teil2.pdf gesamt.pdf
```

`-title -author -subject -keywords -creator -producer`. Leerer Wert löscht den Eintrag. Nicht genannte Einträge behalten, was das erste Dokument hatte.

Seit 0.9.4.51 stehen die Werte in `/Info` **und** im XMP. Vorher nur `/Info` — Acrobat (XMP bevorzugt) zeigte weiter den Titel von Teil 1. Das Paket wird **editiert, nicht neu gebaut**, damit Erweiterungsschemata (etwa Factur-X `fx:`) erhalten bleiben.

ISO 19005-1 Klausel 6.7.3 verlangt Gleichheit von `/Info` und XMP. Prüfen gegen **1b**, nicht 2b:

PDF/A-2/3 haben die Regel fallen lassen. Dieselbe inkonsistente Datei: veraPDF 1b FAIL, 2b PASS.

12.3 Getaggt mischen

Beide Eingaben getaggt: ein Strukturbaum, Elemente des zweiten Dokuments an das erste gehängt. Seitenmarker bleiben.

Getagt plus ungetagt geht nicht sauber. pdf4tcl schreibt gültiges PDF und hängt an `::pdf4tcl::warnings`:

- getagt + ungetagt → spätere Seiten außerhalb des Baums
- ungetagt + getagt → Struktur des zweiten Dokuments fällt weg

Vorher alle Teile gleich taggen.

12.4 Formulare (0.9.4.44)

Die Feldliste ist die Vereinigung. Vorher überlebte nur das Formular des ersten Dokuments: Widgets der anderen saßen auf den Seiten, kein Leser bot sie zum Füllen an.

Namen werden nicht eindeutig gemacht. Zwei Felder desselben Namens sind ein Feld mit einem gemeinsamen Wert (ISO 32000-1 12.7.3.2). Kollision steht in `::pdf4tcl::warnings`.

12.5 XRef-Streams

Objektverzeichnis als Stream (PDF 1.5+): ISO 19005-1 verbietet die Form, PDF/A-2 und -3 verlangen sie. Seit 0.9.4.49/50 liest catPdf beides, inklusive Object-Streams.

Eingabe	Zusammenfügen
gewöhnliches PDF aus pdf4tcl	ja
PDF/A-1b	ja
PDF/A-2b, -3b, -3a	seit 0.9.4.49
Objekte in Containern	seit 0.9.4.50

`qpdf --object-streams=generate` ohne `--newline-before-endstream` verletzt ISO 19005-2 6.1.7.1. Die **Eingabe** fällt in veraPDF durch, also auch das Ergebnis — pdf4tcl reicht Streams unverändert durch.

12.6 Grenzen

- Fontdictionaries bleiben verdoppelt; identische Font-Streams werden geteilt.
- Katalog- und Pages-Objekte angehängter Dokumente bleiben als tote Objekte (wenige hundert Byte).
- Waisen (unerreichbare `/Info` der angehängten Dateien) fallen seit 0.9.4.50 weg.

12.7 Zusammenfassung

- `catPdf ?optionen? eingabe … ausgabe`.
- Titel seit 0.9.4.48, XMP seit 0.9.4.51.

- Struktur seit 0.9.4.40, Formulare seit 0.9.4.44, XRef-Stream seit 0.9.4.49.

Nächstes Kapitel: PDF/A

13 PDF/A

13.1 Einführung

-pdfa am Konstruktor setzt die Konformitätsstufe. Ohne eingebettete Schrift fällt die Datei in veraPDF durch — Base-14 reicht nicht.

13.2 Stufe B

```
pdf4tcl::loadBaseTrueTypeFont BaseDejaVu /pfad/DejaVuSans.ttf
pdf4tcl::createFont BaseDejaVu Body iso8859-1

set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 50 \
        -pdfa 2b]
$pdf metadata -title "Archivprobe" -author "pdf4tcl"
$pdf startPage
$pdf setFont 12 Body
$pdf text "Diese Datei behauptet PDF/A-2b." -x 0 -y 24
$pdf endPage
$pdf write -file archiv-2b.pdf
$pdf destroy
```

Optional: `-pdfa-icc /pfad/srgb.icc`. Sonst sucht pdf4tcl ein Profil.

Stufe	Typischer Einsatz
1b / 1a	älteres Archiv; keine Transparenz; keine eingebetteten Dateien
2b / 2a	modernes Archiv; XRef-Stream; PDF 1.7
3b / 3a	plus eingebettete Dateien (Factur-X / ZUGFeRD)

1b schreibt `%PDF-1.4` und klassische xref-Tabelle. 2b/3b: XRef-Stream und `%PDF-1.7` (seit 0.9.4.22).

13.3 Stufe U (seit 0.9.4.56)

Zwischen B und A: Erscheinung erhaltbar **und** Text als Unicode extrahierbar, ohne Tagging.

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -pdfa 3u]
```

Kein `1u`: die Stufe kam mit Teil 2. CID-Schriften tragen ToUnicode; was B mit eingebetteter Schrift besteht, besteht U. Einsatz: Archiv ohne Strukturbaum, typisch Scan plus OCR.

13.4 Stufe A (seit 0.9.4.41)

A verlangt getaggtes PDF und Dokumentsprache. Beides wird bei `finish` geprüft; fehlt eines, entsteht keine Datei mit falschem `pdfaid:conformance A`:

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -pdfa 3a]
$pdf tagged 1 -lang de-DE -ua 1
$pdf metadata -title "Zugängliches Archiv"
$pdf viewerPreferences -displaydoctitle 1
$pdf startPage
$pdf setFont 12 Body
$pdf tagText H1 "Titel" -x 0 -y 24
$pdf tagBegin P
$pdf text "Getaggter Fließtext." -x 0 -y 50
$pdf tagEnd
```

A prüft nicht, ob das Tagging sinnvoll ist — nur dass es da ist. Jedes Absatz `/P` und jede Überschrift `/H1` validiert und sagt einem Leser nichts Nützliches.

13.5 Fallen

- Standardschrift unter `-pdfa`: Warnung seit 0.9.4.41, Datei fällt in veraPDF durch.
- `setAlpha` unter 1 mit PDF/A-1: Warnung seit 0.9.4.22.
- Eingebettete Dateien bei PDF/A-1: Fehler. Factur-X: `3b`/`3a`.
- Häkchen: Vektor unter Konformität, siehe Formulare erzeugen.

```
verapdf -f 2b archiv-2b.pdf
verapdf -f 3a archiv-3a.pdf
verapdf -f ua1 archiv-3a.pdf
```

13.6 Zusammenfassung

- `-pdfa 1b|2b|3b|1a|2a|3a|2u|3u`, immer mit eingebetteter Schrift.
- A: `tagged 1 -lang …` vor `finish`.
- U: Unicode ohne Strukturbaum.

Nächstes Kapitel: Ebenen

14 Ebenen

14.1 Einführung

Optionale Inhalte (OCG) schaltet der Leser im Viewer an und aus: Debug-Gitter, Briefkopf, Wasserzeichen. Ebenen gelten dokumentweit; `beginLayer` / `endLayer` umschließen das Zeichnen auf einer Seite.

14.2 Rezept

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1]
set gitter [$pdf addLayer "Debuggitter" -visible 0]
set rumpf [$pdf addLayer "Rumpf" -visible 1]

$pdf startPage

$pdf beginLayer $gitter
```

```
$pdf setStrokeColor 0.85 0.85 0.85
for {set x 0} {$x <= 500} {incr x 50} {
    $pdf line $x 0 $x 700
}
$pdf endLayer

$pdf beginLayer $rumpf
$pdf setFont 12 Helvetica
$pdf text "Sichtbarer Rumpf" -x 50 -y 50
$pdf endLayer

$pdf endPage
$pdf write -file ebenen.pdf
$pdf destroy
```

`addLayer` gibt eine ID für `beginLayer` / `endLayer` und für `-layer` an Annotationen und Formularen.

14.3 Optionen (seit 0.9.4.63)

```
$pdf addLayer "Vordruck" -print 0
$pdf addLayer "Detail"   -zoom {2.0 {}}
$pdf addLayer "Kopf DE"  -group kopf
set liste [$pdf layers]
```

Option	Wirkung
-visible	Anfangszustand (Default 1)
-print 0	auf dem Schirm, nicht auf Papier
-export	Zustand beim Speichern in ein Format ohne OCG
-zoom {min max}	nur in diesem Maßstab an; leer = unbegrenzt
-locked	Viewer darf nicht umschalten
-group name	höchstens eine Ebene der Gruppe sichtbar

`-print 0` schreibt `/Usage << /Print << /PrintState /OFF >> >>`. ISO 32000-1 8.11.4.4: Usage-Dictionaries gelten für **interaktive** konforme Leser, nicht für Endform-Ausgabe. Druck aus dem Viewer ehrt es; ein RIP nicht, und das ist konform. Wo es nicht schiefgehen darf, die Grafik gar nicht in die Datei legen.

Sobald eine Ebene `/Usage` trägt, bekommt die Default-Konfiguration ein `/AS`-Array — ohne das wendet kein Viewer `/Usage` an.

14.4 Zählen

`endLayer` ohne `beginLayer` ist Fehler (früher eine verirrte EMC). `endPage` lehnt eine Seite mit offener Ebene ab (ISO 32000-1 14.6).

14.5 Formulare

Annotationen liegen außerhalb des Content-Streams. `beginLayer` erreicht sie nicht. `-layer` an `addForm` schreibt `/OC` ins Annotations-Dictionary.

14.6 Zusammenfassung

- `addLayer`, dann `beginLayer` / `endLayer` auf der Seite.
- `-print 0` und `-zoom` über `/Usage`; `/AS` entsteht automatisch.
- Offene Ebene am Seitenende ist Fehler.

Nächstes Kapitel: Metadaten

15 Metadaten

15.1 Einführung

Titel, Autor, Betreff stehen zweimal: im klassischen `/Info` und im XMP-Paket, auf das der Katalog zeigt. `metadata` schreibt beide. Acrobat liest bevorzugt XMP.

15.2 Setzen

```
$pdf metadata \
        -title "Jahresbericht 2026" \
        -author "Muster GmbH" \
        -subject "Kennzahlen" \
        -keywords "Bericht, 2026" \
        -creator "bericht.tcl" \
        -producer "pdf4tcl"
```

Unbekannte Option:

```
unknown metadata option "-foo"
```

Leere Werte nach `string trim` werden ignoriert. Datum:

```
$pdf metadata -creationdate 0 -moddate 0
```

`0` bedeutet "jetzt" (`clock seconds`). Sonst ein PDF-Datum, das `_ValidatePdfDate` akzeptiert.

15.3 Viewer

```
$pdf viewerPreferences -displaydoctitle 1 -pagemode UseOutlines
```

`-displaydoctitle 1` zeigt den Dokumenttitel in der Titelleiste statt des Dateinamens. PDF/A-A und PDF/UA erwarten das oft.

Weitere Schalter: `-pagelayout`, `-hidetoolbar`, `-fitwindow`, `-centerwindow`, `-direction L2R|R2L`.

15.4 Anhänge

```
$pdf addEmbeddedFile "notizen.txt" \
        -contents "Beilage\n" \
        -mimetype "text/plain" \
        -description "Nebennotizen" \
        -afrelationship Data
```

`-afrelationship`: `Alternative`, `Data`, `Source`, `Supplement`, `Unspecified`. Verboten unter `-pdfa 1b` (Fehler). PDF/A-3b trägt die Datei automatisch in `/AF`. Factur-X: Methode `facturx`, nicht das Paket von Hand bauen.

15.5 Verschlüsselung

Seit 0.9.4.67 läuft auch der XMP-Stream über Encrypt, wenn die Datei `/EncryptMetadata true` setzt. Bis 0.9.4.66 stand der Titel im Klartext in einer angeblich verschlüsselten Datei. `qpdf --check` sieht das nicht.

15.6 catPdf

Zusammenfügen übernimmt den Katalog des ersten Teils. Titel der Gesamtdatei mit `catPdf -title …` setzen, sonst bleibt der Titel von Teil 1 — auch in XMP seit 0.9.4.51. Siehe catPdf.

15.7 Zusammenfassung

- `metadata` füllt `/Info` und XMP.
- `viewerPreferences -displaydoctitle 1` für Archiv und UA.
- Anhänge: PDF/A-3, nicht PDF/A-1.

Nächstes Kapitel: gsave und Text

16 gsave und Text

16.1 Einführung

`text` lässt das Textobjekt offen, damit mehrere Aufrufe ein `BT` teilen. `q` (`gsave`) und `Q` (`grestore`) sind Grafikoperatoren. ISO 32000-1 Abschnitt 8.2 erlaubt zwischen `BT` und `ET` nur Textoperatoren.

16.2 Die Falle bis 0.9.4.61

`gsave` und `grestore` schrieben `q` / `Q` gerade ins offene Textobjekt:

```
BT … q … ET Q
```

Poppler, MuPDF und pdftotext zeichneten den ungültigen Stream trotzdem. **Adobe Reader zeigte eine leere Seite ohne Meldung.** `pdfinfo` meldete eine gesunde Datei.

Gefunden über `pdf4tcllib::labels::render` mit `-clip 1`. `tests/nesting.test` prüft die Klasse, nicht einen String.

16.3 Seit 0.9.4.62

Beide Methoden rufen `EndTextObj` **vor** `q` / `Q`. Am Aufruf ändert sich nichts. Wer erzeugte Dateien byteweise vergleicht, sieht `q` und `Q` außerhalb von `BT`/`ET`, und ein `ET` kann erscheinen, wo keines war.

```
$pdf setFont 12 Helvetica
$pdf text "Zeile eins" -x 50 -y 50
$pdf gsave
$pdf clip 40 40 200 80
$pdf rectangle 40 40 200 80 -filled 1
$pdf grestore
$pdf text "Zeile zwei" -x 50 -y 80
```

Nach `gsave` beginnt der nächste `text` ein neues Textobjekt. Das ist richtig, nicht ein Rückschritt.

16.4 Andere Methoden

`clip`, `rectangle` und der Linienweg schließen das Textobjekt selbst bzw. über `DrawLine`. Nur `gsave` / `grestore` taten das nicht — bis 0.9.4.62.

Transformationen (`translate`, `rotate`, `scale`) gehören zwischen `gsave` und `grestore`. Text mit `Tm` folgt `cm` nicht; siehe Grafik. Trotzdem zuerst den Grafikzustand retten, sonst bleibt die Matrix für alles Weitere schief.

16.5 Regel

Nach `text` und vor jeder Grafikoperation, die `q`, `cm`, `W` oder Pfadoperatoren schreibt: entweder eine Zeichenmethode nutzen, die `EndTextObj` selbst aufruft, oder explizit `gsave`. Seit 0.9.4.62 reicht `gsave`.

Adobe als Prüfleser behalten. Eine Seite, die in Poppler voll ist und in Acrobat leer, ist oft genau dieser Stream.

16.6 Tagged Text seit 0.9.4.69

`TagEnsureMC` schreibt `BDC`/`EMC` außerhalb von `BT`/`ET` und kann dafür ein offenes Textobjekt schließen. Stand der Aufruf zwischen `Tm` und `Tj`, lag `Tj` außerhalb:

```
BT / Tm / ET / << /MCID n >> BDC / (…) Tj / EMC
```

ISO 32000-1 9.4: Textausgabe gehört zwischen `BT` und `ET`. Adobe zeichnet sie außerhalb nicht; Poppler, MuPDF und Chrome tun es. Seit 0.9.4.69 kommt die Auszeichnung **vor** `BeginTextObj`. Schließt `TagEnsureMC` ein offenes Textobjekt, folgt `Tm` neu (`posSet`).

16.7 Zusammenfassung

- `text` hält `BT` offen.
- `gsave` / `grestore` schließen `BT` vor `q` / `Q` (0.9.4.62).
- Tagged: Auszeichnung vor `BT`, sonst `Tj` draußen (0.9.4.69).

- Adobe: leere Seite ohne Meldung, wenn `q` in `BT` landet oder `Tj` außerhalb von `BT/ET`.

Nächstes Kapitel: CID-Subset und CMap

17 CID-Subset und CMap

17.1 Einführung

Seit 0.9.4.57 schreibt pdf4tcl bei CID nur die benutzten Glyphen (TrueType). Die Teilmenge muss eine gültige TrueType-Datei sein: Pflichttabellen, cmap, Prüfsummen. Bis **0.9.4.67** fehlten cmap und echte Tabellen-Prüfsummen. **0.9.4.68** setzt beides.

Aufrufer ändern nichts. `createFontSpecCID` und `setFont` bleiben. Dieses Kapitel erklärt, warum Adobe eine leere Seite zeigte und warum `letzteGlyphe numGlyphs-1` ist, nicht `0xFFFE`.

17.2 cmap ist Pflicht

cmap steht in den Required Tables (Apple TrueType, OpenType). Bis 0.9.4.67 strich `CIDRebuildTtf` sie: Identity-H adressiert Glyphen direkt. Das gilt für die PDF-Seite — und deshalb merkte es niemand:

- Viewer mit `/CIDToGIDMap /Identity` zeichneten die Glyphen
- `pdftotext` extrahierte den richtigen Text
- `qpdf` fand keinen Syntaxfehler
- **Adobe Reader zeigte eine leere Seite**

Die Teilmenge war einfach keine gültige Schrift. Ob Reader wirklich aus dieser Tabelle malt, ist nicht geprüft. Die PDF-Norm sagt, für eingebettetes CIDFontType2 gelte CIDToGIDMap. Die wahrscheinliche Ursache: Reader prüft das Fontprogramm. Die Korrektur stützt sich auf die Pflicht der cmap, nicht auf die Vermutung.

17.3 Identity, nicht Unicode

Eine Viewer-Schleife durch die cmap läse die Glyph-ID als Zeichencode. Glyph `0x5A` ist nicht U+005A. Beide Wege müssen dasselbe Glyph finden. Deshalb schreibt CIDIdentityCmap eine Identity-Abbildung, nicht die originale Unicode-cmap.

Zwei Segmente, Format 4:

- `0.. letzteGlyphe` bildet auf sich selbst ab
- `0xFFFF` ist der vorgeschriebene Wächter und zeigt auf Glyph 0

17.4 letzteGlyphe ist numGlyphs-1

Ein cmap-Eintrag darf nicht auf ein Glyph zeigen, das es nicht gibt. Ein früherer Entwurf führte das erste Segment bis `0xFFFE`. Bei DejaVuSans zeigten dann 59282 Codes ins Leere; fontTools lehnte die Schrift ab. `cidfont-11.2` hält beide Enden.

`CIDIdentityCmap` begrenzt: unter 0 wird 0, über `0xFFFE` wird `0xFFFE`. Der Aufruf aus `CIDRebuildTtf`:

```
CIDIdentityCmap [expr {$numGlyphs - 1}]
```

`numGlyphs` steht in `maxp` bei Offset 4.

17.5 Prüfsummen (0.9.4.68)

Zwei Stellen:

checkSumAdjustment in `head` wird auf 0 gesetzt, bevor die Tabellensumme läuft. Der Originalwert gehört zur **Originalschrift** und lügt in einer Teilmenge. Die Norm berechnet die Tabellensumme von `head` mit diesem Feld auf 0. Der Endwert kommt ganz am Schluss, wenn die Datei steht.

Prüfsumme je Tabelle. Bis 0.9.4.67 stand hier 0. Die erzeugte Datei war keine gültige TrueType-Datei — so deutlich, dass `loadBaseTrueTypeFont` sie mit `invalid TTF file checksum` ablehnte. `MakeTTFSubset` (der andere Schneider) macht das seit jeher richtig; nur der CID-Weg nicht.

Layout-Tabellen (GPOS, GSUB, GDEF, kern, MATH, FFTM, DSIG) bleiben draußen: nicht Pflicht, und pdf4tcl hat den Inhalt beim Laden schon in `BFA`.

17.6 CFF

OpenType/CFF wird nicht subsettet. Es gibt eine Warnung, die ganze Face wandert ins PDF. Derselbe cmap-Fix gilt dort nicht.

17.7 Zusammenfassung

- Bis 0.9.4.67: keine cmap im CID-Subset; Adobe leer, andere Viewer voll.
- 0.9.4.68: `CIDIdentityCmap` mit `numGlyphs-1`, nicht `FFFE`.
- Tabellen-Prüfsummen und `checkSumAdjustment` 0, dann Endwert.
- API unverändert.

Nächstes Kapitel: Fallstudie: Bericht

18 Fallstudie: Bericht

18.1 Einführung

Ein einseitiger Bericht: CID-Schrift, Kopf, drei Kennzahlen, ein Formularfeld, Stempel. Die Teile kommen aus den vorigen Kapiteln, hier in einem Ablauf.

18.2 Schrift und Dokument

```
package require pdf4tcl 0.9.4.69

pdf4tcl::loadBaseTrueTypeFont BaseDejaVu \
        /usr/share/fonts/truetype/dejavu/DejaVuSans.ttf
pdf4tcl::createFontSpecCID BaseDejaVu Bericht

```

```
set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 48 \
        -userpassword "lesen" -ownerpassword "setzen" \
        -permissions {print}]
$pdf metadata -title "Monatsbericht März 2026" -author "Muster GmbH"
$pdf startPage
$pdf setFont 16 Bericht
```

Verschlüsselung plus CID: seit 0.9.4.67 lesbar. User darf drucken, nicht kopieren.

18.3 Kopf und Kennzahlen

```
$pdf text "Monatsbericht März 2026" -x 0 -y 8
$pdf setFont 10 Bericht
$pdf text "Muster GmbH — intern" -x 0 -y 28

$pdf setStrokeColor 0.2 0.2 0.2
$pdf setLineWidth 0.5
$pdf line 0 40 500 40

set kasten {{Umsatz 128} {Aufträge 41} {Offen 7}}
set x 0
foreach paar $kasten {
    lassign $paar label wert
    $pdf setFillColor 0.93 0.93 0.93
    $pdf rectangle $x 56 150 48 -filled 1
    $pdf setFillColor 0 0 0
    $pdf text $label -x [expr {$x+10}] -y 64
    $pdf setFont 18 Bericht
    $pdf text $wert -x [expr {$x+10}] -y 84
    $pdf setFont 10 Bericht
    incr x 170
}
```

Nach grauer Füllung die Füllfarbe auf Schwarz zurücksetzen, sonst erbt der Text Grau.

18.4 Feld und Stempel

```
$pdf text "Freigabe:" -x 0 -y 130
$pdf addForm text 80 120 200 16 -id freigabe -maxlen 24

$pdf addAnnotStamp 400 500 90 28 -name Approved -color {0.2 0.5 0.2}
```

`setFont` stand schon. Ohne Schrift bliebe der Stempel in PDFium leer (Annotationen).

```
$pdf endPage
$pdf write -file bericht.pdf
$pdf destroy
```

18.5 Später füllen

`fillForms` und `getForms` lesen verschlüsselte Dateien nicht (kein `-password`). Auf der AES-`bericht.pdf` von oben schlägt `fillForms` mit `no such field(s) … freigabe` fehl. Für

den Fill-Schritt dieselbe Zeichnung **ohne** `-userpassword`/`-ownerpassword` schreiben:

```
# Konstruktor ohne Kennwörter, sonst wie oben …
$pdf write -file bericht-offen.pdf
$pdf destroy
pdf4tcl::fillForms bericht-offen.pdf bericht-frei.pdf \
        {freigabe "Müller, 28.03."}
```

Einfaches Textfeld: Appearance wird neu gebaut (0.9.4.64). NeedAppearances bleibt aus, der Rahmen eines unberührten Kästchens — hier keines — ginge nicht verloren.

Prüfen:

```
qpdf --check bericht-frei.pdf
pdftotext bericht-frei.pdf -
```

18.6 Was bewusst fehlt

- Seitenumbruch und Tabelle: `pdf4tcllib`
- PDF/A: Base-14 weglassen, `-pdfa 2b`, `metadata` bleibt
- Zweite Seite per `catPdf` mit Anhang
- Comb-Feld ohne MaxLen: nicht füllen und Appearance erwarten

Das ist der Kern, den die Howtos zerlegen. Hier steht er als eine Anwendung.

18.7 Zusammenfassung

- CID laden, Dokument mit AES, Kopf zeichnen, Feld, Stempel, schreiben.
- fillForms für das Textfeld; Stempel nur nach setFont.
- Layout über Seiten: nicht in diesem Paket.

Nächstes Kapitel: Fallen

19 Fallen

Kurze Liste. Begründung in den Kapiteln.

19.1 Text und Schrift

- Helvetica **-Oblique**, Times **-Italic**.
- Base-14: alles außerhalb Latin-1 wird?. `getSubstCount` unter 8.6 oft 0 trotz Fragezeichen.
- Kodierte Teilmenge:? (63) in die Liste, sonst Fallback auf Slot 0.
- `drawTextBox -newyvar`: Aufrufer-Koordinaten seit 0.9.4.44; alte `pageHeight`-Korrektur entfernen.
- Weiches Trennzeichen seit 0.9.4.61 ohne Glyph und ohne Breite.
- CFF/OpenType: CID subsettet nicht, Warnung, ganze Face.

19.2 Grafik

- Farbe 0..1, nicht 0..255 (Fehler seit 0.9.4.39).
- `setBgColor` ist nicht der Seitengrund.
- q in offenem BT: Adobe leer ohne Meldung. Seit 0.9.4.62 schließt `gsave` das Textobjekt.

- Tagged vor 0.9.4.69: `Tj` außerhalb von `BT`/`ET`, Adobe leer, Poppler voll. Auszeichnung vor `BeginTextObj`.
- Text folgt `cm` nicht.

19.3 Canvas

- `$pdf canvas`: `update`, dann `-bbox`.
- tko-Matrix flach `{a b c d tx ty}`, tkpath verschachtelt.
- `-fontmap`: Canvas ganze Spezifikation, tko nur Familie.
- window oft weg.

19.4 Formulare und Annotationen

- fillForms baut Appearance für einfaches Textfeld (0.9.4.64).
- Comb ohne MaxLen, Kennwort, Auswahl: alter Stream.
- NeedAppearances nur dort, sonst PDFium löscht unberührte Kästchen.
- Feldname `person.city`, nicht `city`.
- Stempel nach `setFont`, sonst PDFium leer.
- Dynamisches XFA: fillForms lehnt ab.

19.5 Ausgabe

- CID plus Encrypt: Dateien vor 0.9.4.67 neu bauen.
- CID-Subset vor 0.9.4.68: Adobe leer (keine cmap / Prüfsumme 0).
- PDF/A: Schrift einbetten; Base-14 nur Warnung, kein gültiges Archiv.
- catPdf: Titel setzen, sonst Titel von Teil 1; XMP seit 0.9.4.51.
- Offene Ebene am `endPage`: Fehler.

Nächstes Kapitel: API

20 API

Kurzreferenz pdf4tcl 0.9.4.69, Links im Dokument aus 0.9.4.72. Optionen unvollständig; die Manpage bleibt die Norm.

20.1 Paket und Objekt

```
package require pdf4tcl 0.9.4.69
set pdf [::pdf4tcl::new %AUTO% -paper a4 -orient 1 -margin 50]
$pdf startPage
$pdf endPage
$pdf write -file out.pdf
$pdf destroy
```

Konstruktor (Auswahl): `-paper -orient -landscape -margin -unit -compress -cmyk -pdfa -pdfa-icc -markstyle -userpassword -ownerpassword -encversion -permissions`.

`write -file` / `-chan`. Warnungen: `::pdf4tcl::warnings`.

20.2 Text und Schrift

```
$pdf setFont 12 Helvetica
$pdf text "Zeile" -x 0 -y 24
$pdf drawTextBox x y w h $txt -align left -newyvar nextY
$pdf getStringWidth $txt
$pdf getStringWidth $txt -font Helvetica -size 12
$pdf getSubstCount

pdf4tcl::loadBaseTrueTypeFont Base datei.ttf
pdf4tcl::createFont Base Body iso8859-1
pdf4tcl::createFontSpecEnc Base Body $codepoints
pdf4tcl::createFontSpecCID Base Uni
```

20.3 Grafik

```
$pdf setFillColor red
$pdf setStrokeColor 0 0 0
$pdf setLineWidth 1
$pdf setAlpha 0.5
$pdf line x1 y1 x2 y2
$pdf rectangle x y w h -filled 1
$pdf circle x y r
$pdf gsave
$pdf clip x y w h
$pdf linearGradient x1 y1 x2 y2 c1 c2
$pdf translate dx dy
$pdf rotate winkel
$pdf scale sx sy
$pdf grestore
```

20.4 Bilder und Canvas

```
set id [$pdf addImage datei.png -id logo]
$pdf putImage $id x y -width 120
$pdf canvas .c -bbox $bb -x 50 -y 50 -width 400 -height 300 \
        -sticky nw -fontmap {DocText Uni}
```

20.5 Formular, Annotation, Ebene

```
$pdf addForm text x y w h -id name -init "" -maxlen 8 -comb 1
pdf4tcl::getForms in.pdf
pdf4tcl::fillForms in.pdf out.pdf {name Wert}
pdf4tcl::exportForms in.pdf out.fdf

$pdf addAnnotStamp x y w h -name Approved
$pdf addAnnotNote x y w h -content "…"
$pdf addAnnotLine x1 y1 x2 y2
$pdf hyperlinkAdd x y w h url                   ;# Adresse
$pdf destinationAdd name ?-y y? ?-x x?          ;# Ziel (0.9.4.72)
$pdf linkAdd x y w h name                       ;# Sprung (0.9.4.72)
```

```
set l [$pdf addLayer "Gitter" -visible 0 -print 0]
$pdf beginLayer $l
$pdf endLayer
```

20.6 Zusammenfügen, Metadaten, PDF/A

```
pdf4tcl::catPdf -title "Gesamt" a.pdf b.pdf out.pdf
$pdf metadata -title "…" -author "…"
$pdf viewerPreferences -displaydoctitle 1
$pdf tagged 1 -lang de-DE -ua 1
$pdf addEmbeddedFile name.txt -contents $bytes -mimetype text/plain
```

Konstruktor `-pdfa 1b|2b|3b|1a|2a|3a|2u|3u`. `-encversion` 4 AES-128, 5 AES-256.

Nächstes Kapitel: Glossar

21 Glossar

21.1 A

AES-128 / AES-256
: Verschlüsselung über `-encversion` 4 bzw. 5. User öffnet, Owner hat volle Rechte.

Appearance-Stream
: Zeichnung eines Felds oder einer Annotation. Ohne ihn zeigt PDFium Stempel nicht. fillForms baut ihn für einfaches Textfeld neu.

21.2 B

Base-14
: Vierzehn Standardschriften, nicht einbettbar. Nur WinAnsi.

21.3 C

catPdf
: PDFs aneinanderhängen. Struktur seit 0.9.4.40, Formulare seit 0.9.4.44.

CID-Schrift
: Type-0 mit Identity-H, Unicode über TrueType. Subset seit 0.9.4.57.

CIDIdentityCmap
: Identity-cmap der Teilmenge seit 0.9.4.68. `letzteGlyphe = numGlyphs-1`.

cmap
: Pflicht-TrueType-Tabelle Zeichen nach Glyph. Bis 0.9.4.67 im CID-Subset gestrichen.

21.4 N

NeedAppearances
: Flagge am AcroForm: Viewer soll Erscheinungen neu bauen. fillForms setzt sie nur, wo der Stream alt bleibt.

21.5 O

OCG
Optional Content Group, Ebene. `addLayer` / `beginLayer` / `endLayer`.

orient
`-orient 1` (Default): Y von oben.

21.6 P

PDF/A
Archivformat. Schrift einbetten. Stufen B, U, A; Teile 1–3.

PDFium
Viewer in Chrome und Edge. Stempel und Linie ohne Appearance unsichtbar. NeedAppearances leert Off-Kästchen.

21.7 T

Textobjekt
BT … ET. `text` hält es offen. `gsave` schließt es seit 0.9.4.62 vor `q`.

ToUnicode
Abbildung Glyph nach Unicode für Extraktion. Base-14 seit 0.9.4.9 für WinAnsi; CID immer.

21.8 W

WriteObjectDirect
Schreibt ein Objekt über Encrypt. Pflicht für CID-Schriftobjekte seit 0.9.4.67.

21.9 X

XMP
Metadatenpaket am Katalog. `metadata` und `catPdf -title` schreiben `/Info` und XMP (catPdf seit 0.9.4.51).

22 Fehlermeldungen

Häufige Meldungen und die übliche Ursache.

22.1 no font set

`addForm` für `combobox`, `listbox`, `pushbutton` oder `signature` vor `setFont`. Textfeld und Kästchen dürfen vorher kommen. Seit 0.9.4.63; früher `can't read "FontsAttrs(,specialencoding)"`.

22.2 unknown color: "1" -- expected a name like "red", "#rrggbb", …

Farbe kein Triplett 0..1, kein Hex, kein Tabellenname. `1` allein ist ungültig. Seit 0.9.4.63 mit Beispiel in der Meldung.

22.3 unknown color / invalid … component

Komponente außerhalb 0.0.. 1.0 (seit 0.9.4.39). `255 0 0` durch `1 0 0` oder `#ff0000` ersetzen.

22.4 createFontSpecCID: base font '...' not loaded

`loadBaseTrueTypeFont` fehlt oder der Basisname weicht ab.

22.5 invalid TTF file checksum

Beschädigte oder unvollständige TrueType-Datei. CID-Subsets vor 0.9.4.68 trugen Tabellen-Prüfsumme 0 und fielen hier durch, wenn man sie erneut lud.

22.6 fillForms: no such field(s)

Name nicht im Formular. Seit 0.9.4.64 volle Kette: `person.city`, nicht `city`. Tippfehler nicht ignorieren — sonst leeres Formular ohne Erklärung.

22.7 fillForms: "..." is not an option of "..."

Auswahlwert weder Export noch Label. Leerer Wert ist erlaubt. Seit 0.9.4.65.

22.8 unknown form type ...

Typ nicht in `text checkbutton combobox listbox password radiobutton pushbutton signature`. Alias: `checkbox` für `checkbutton`.

22.9 unknown metadata option "..."

Nur `-author -creator -keywords -producer -subject -title -creationdate -moddate`.

22.10 papersize "..." is unknown

Kein ISO-Name und keine Zweierliste `{breite höhe}`.

22.11 rotation ... not a multiple of 90

`-rotate` nur Vielfache von 90.

22.12 endLayer without beginLayer / layer still open

Ebene nicht paarig. endPage mit offener Ebene ist Fehler (0.9.4.63, ISO 32000-1 14.6).

22.13 Adobe: leere Seite, keine Meldung

Kein throw. q in BT (vor 0.9.4.62), CID-Subset ohne cmap (vor 0.9.4.68) oder tagged `Tj` außerhalb von BT/ET (vor 0.9.4.69). In Poppler oft voll. Siehe gsave und Text und CID-Subset und CMap.

22.14 qpdf: inflate: incorrect header check

Verschlüsseltes PDF mit CID aus 0.9.4.66 oder früher. Neu bauen mit 0.9.4.67+. Siehe Verschlüsselung.

22.15 PDF/A: the standard font Helvetica has no embeddable font program

Warnung in `::pdf4tcl::warnings`, kein Abbruch. veraPDF fällt durch. TrueType laden.

22.16 pdf4tcl: linkAdd target(s) never defined with destinationAdd: …

Warnung auf stderr beim Schreiben, kein Abbruch. Ein `linkAdd` nennt einen Namen, den kein `destinationAdd` angelegt hat; der Link bleibt wirkungslos. Namen angleichen. Siehe Annotationen.

22.17 destination "…" already defined

Derselbe Name zweimal an `destinationAdd`. Namen sind je Dokument eindeutig.

22.18 invalid destination name "…": must match [A-Za-z0-9._-]+

Leerzeichen, Umlaut oder Sonderzeichen im Namen eines Ziels oder Links.

Ende des Anhangs. Einstieg: pdf4tcl verstehen

23 Lizenz


```
Copyright (c) 2026 Gregor Ebbing

Text: Creative Commons Namensnennung - Weitergabe unter gleichen
Bedingungen 4.0 International (CC BY-SA 4.0)
Codebeispiele des Autors: MIT-Lizenz (unten)

---

Text

Dieses Buch ist unter der Creative-Commons-Lizenz
"Attribution-ShareAlike 4.0 International" (CC BY-SA 4.0) lizenziert.

Sie dürfen das Material in jedwedem Format oder Medium vervielfältigen
und weiterverbreiten sowie es remixen, verändern und darauf aufbauen,
auch zu kommerziellen Zwecken, unter den folgenden Bedingungen:

* Namensnennung: Sie müssen angemessene Urheber- und Rechteangaben
  machen, einen Link zur Lizenz beifügen und angeben, ob Änderungen
  vorgenommen wurden.

* Weitergabe unter gleichen Bedingungen: Wenn Sie das Material
  remixen, verändern oder anderweitig direkt darauf aufbauen, dürfen
  Sie Ihre Beiträge nur unter derselben Lizenz wie das Original
  verbreiten.

Verbindlich ist der vollständige Lizenztext:
https://creativecommons.org/licenses/by-sa/4.0/legalcode.de
Englische Originalfassung:
https://creativecommons.org/licenses/by-sa/4.0/legalcode

---
```

```
Codebeispiele

Die Codebeispiele des Autors (in den Codeblöcken des Buchs und als
Skripte daneben) stehen zusätzlich unter der MIT-Lizenz. Übernommene
Codebeispiele und zitierte Texte aus anderen Projekten (Tcl/Tk,
pdf4tcl und andere) behalten die Lizenz ihres Originals; die Quelle ist
jeweils angegeben.

MIT License

Copyright (c) 2026 Gregor Ebbing

Permission is hereby granted, free of charge, to any person obtaining
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

Stichwortverzeichnis

-

-pdfa 26

A

AES 23
Appearance-Stream 16

C

canvas 14
catPdf 24
CID-Schrift 6, 10
CIDIdentityCmap 32

D

destinationAdd 22
Dokument 7

E

Ebene 8

F

fillForms 18

H

hyperlinkAdd 22

L

linkAdd 22

M

Matrix 15

N

NeedAppearances 19

O

OCG 27

P

pdf4tcl 6

S

Standardschriften 9

T

Textobjekt 9, 30